Skill

Task 1 : To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
vennela_07@VennelaPC:~$ cd ~
mkdir command_history_project
cd command_history_project
vennela_07@VennelaPC:~/command_history_project$ nano command_history.c
vennela_07@VennelaPC:~/command_history_project$ gcc -Wall -Wextra command_history.c -o command_history
vennela_07@VennelaPC:~/command_history_project$ ./command_history

--- Command History Menu ---
1. Store Command
2. Display History
3. Retrieve Entry
4. Validate Consistency
5. Exit
Enter your choice: ls
pwd
date
whoami
gcc main.cInvalid input. Enter a number.

--- Command History Menu ---
1. Store Command
2. Display History
3. Retrieve Entry
4. Validate Consistency
5. Exit
Enter your choice: Invalid input. Enter a number.

--- Command History Menu ---
1. Store Command
2. Display History
3. Retrieve Entry
4. Validate Consistency
5. Exit
Enter your choice: Invalid input. Enter a number.

--- Command History Menu ---
1. Store Command
2. Display History
3. Retrieve Entry
4. Validate Consistency
```

```
4. Validate Consistency
5. Exit
Enter your choice: Invalid input. Enter a number.

--- Command History Menu ---
1. Store Command
2. Display History
3. Retrieve Entry
4. Validate Consistency
5. Exit
Enter your choice: mkdir test
Invalid input. Enter a number.

--- Command History Menu ---
1. Store Command
2. Display History
3. Retrieve Entry
4. Validate Consistency
5. Exit
Enter your choice: 3
History is empty.

--- Command History Menu ---
1. Store Command
2. Display History
3. Retrieve Entry
4. Validate Consistency
5. Exit
Enter your choice: 4
History consistency verified.
```

```
--- Command History Menu ---
1. Store Command
2. Display History
3. Retrieve Entry
4. Validate Consistency
5. Exit
Enter your choice: 5
Exiting command history program.
```

GNU nano 7.2 command_history.c

```c
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int count = 0;

void store_command(const char *command)
{
    int i;

    if (count == MAX_HISTORY)
    {
        for (i = 1; i < MAX_HISTORY; i++)
        {
            strcpy(history[i - 1], history[i]);
        }
        count--;
        printf("History full. Oldest command removed.\n");
    }

    strcpy(history[count], command);
    count++;

    printf("Command stored successfully.\n");
}

void display_history(void)
{
    int i;

    if (count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    printf("\nCommand History:\n");

    for (i = 0; i < count; i++)
    {
        printf("%d. %s\n", i + 1, history[i]);
    }
}

void retrieve_entry(void)
{
    int position;
```

```
    printf("Enter history position (1-%d): ", count);

    if (scanf("%d", &position) != 1)
    {
        printf("Invalid input.\n");
        while (getchar() != '\n')
            ;
        return;
    }

    while (getchar() != '\n')
        ;

    if (position < 1 || position > count)
    {
        printf("Invalid history position.\n");
        return;
    }

    printf("Retrieved command: %s\n", history[position - 1]);
}

void validate_history(void)
{
    int i, valid = 1;

    if (count < 0 || count > MAX_HISTORY)
    {
        valid = 0;
    }

    for (i = 0; i < count; i++)
    {
        if (history[i][0] == '\0')
        {
            valid = 0;
            break;
        }
    }

    if (valid)
        printf("History consistency verified.\n");
    else
        printf("History consistency error detected.\n");
}

int main(void)
{
    int choice;
    char command[MAX_COMMAND];
```

```c
    while (1)
    {
        printf("\n--- Command History Menu ---\n");
        printf("1. Store Command\n");
        printf("2. Display History\n");
        printf("3. Retrieve Entry\n");
        printf("4. Validate Consistency\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input. Enter a number.\n");
            while (getchar() != '\n')
                ;
            continue;
        }

        while (getchar() != '\n')
            ;

        switch (choice)
        {
            case 1:
                printf("Enter command: ");

                if (fgets(command, sizeof(command), stdin) == NULL)
                {
                    printf("Input error.\n");
                    return 1;
                }

                command[strcspn(command, "\n")] = '\0';

                if (command[0] == '\0')
                {
                    printf("Command cannot be empty.\n");
                    break;
                }

                store_command(command);
                break;

            case 2:
                display_history();
                break;

            case 3:
                if (count == 0)
                {
                    printf("History is empty.\n");
```

Task 2 : To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
vennela_07@VennelaPC:~/command_history_project$ cd ~
mkdir pipeline_project
cd pipeline_project
vennela_07@VennelaPC:~/pipeline_project$ nano pipeline.c
vennela_07@VennelaPC:~/pipeline_project$ gcc -Wall -Wextra pipeline.c -o pipeline
vennela_07@VennelaPC:~/pipeline_project$ ./pipeline
Enter pipeline (example: ls | wc -l): Pipeline Layout:
Stage 1: ls
Sta
Pipeline Layout:
Stage 1: Pipeline
ge 2: wc -l

Pipeline execution completed.execvp: No such file or directory

Pipeline execution completed.
vennela_07@VennelaPC:~/pipeline_project$ Stage 1: ls
Stage: command not found
vennela_07@VennelaPC:~/pipeline_project$ Stage 2: wc -l
Stage: command not found
vennela_07@VennelaPC:~/pipeline_project$
vennela_07@VennelaPC:~/pipeline_project$ Pipeline execution completed.
Pipeline: command not found
vennela_07@VennelaPC:~/pipeline_project$ ./pipeline
Enter pipeline (example: ls | wc -l): ls | sort | wc -l

Pipeline Layout:
Stage 1: ls
Stage 2: sort
Stage 3: wc
9

Pipeline execution completed.
vennela_07@VennelaPC:~/pipeline_project$ ./pipeline
Enter pipeline (example: ls | wc -l): ls ||
Invalid pipeline: empty command.
vennela_07@VennelaPC:~/pipeline_project$ ls
Makefile  README.md  demo  docs  include  pipeline  pipeline.c  src  tests
```

```
 GNU nano 7.2                                                pipeline.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 5
#define MAX_INPUT 256
#define MAX_ARGS 16

typedef struct {
    char text[MAX_INPUT];
    char *args[MAX_ARGS];
} Command;

int main(void)
{
    Command commands[MAX_COMMANDS];
    char input[MAX_INPUT];
    int n = 0, i, j;
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];
    char *start, *end;

    printf("Enter pipeline (example: ls | wc -l): ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    start = input;

    while (1)
    {
        end = strchr(start, '|');

        if (end != NULL)
            *end = '\0';

        while (*start == ' ' || *start == '\t')
            start++;

        if (*start == '\0')
        {
            printf("Invalid pipeline: empty command.\n");
            return 1;
        }
```

```
        }

        if (n >= MAX_COMMANDS)
        {
            printf("Too many commands. Maximum is %d.\n",
                   MAX_COMMANDS);
            return 1;
        }

        strncpy(commands[n].text, start, MAX_INPUT - 1);
        commands[n].text[MAX_INPUT - 1] = '\0';

        int argc = 0;
        char *token = strtok(commands[n].text, " \t");

        while (token != NULL)
        {
            if (argc >= MAX_ARGS - 1)
            {
                printf("Too many arguments in command.\n");
                return 1;
            }

            commands[n].args[argc++] = token;
            token = strtok(NULL, " \t");
        }

        if (argc == 0)
        {
            printf("Invalid empty command.\n");
            return 1;
        }

        commands[n].args[argc] = NULL;
        n++;

        if (end == NULL)
            break;

        start = end + 1;
    }

    printf("\nPipeline Layout:\n");

    for (i = 0; i < n; i++)
        printf("Stage %d: %s\n", i + 1, commands[i].text);

    for (i = 0; i < n - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return 1;
        }
    }

    for (i = 0; i < n; i++)
    {
        pids[i] = fork();

        if (pids[i] == -1)
        {
```

```c
        {
            perror("fork");
            return 1;
        }

        if (pids[i] == 0)
        {
            if (i > 0)
                dup2(pipes[i - 1][0], STDIN_FILENO);

            if (i < n - 1)
                dup2(pipes[i][1], STDOUT_FILENO);

            for (j = 0; j < n - 1; j++)
            {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            execvp(commands[i].args[0], commands[i].args);
            perror("execvp");
            _exit(1);
        }
    }

    for (i = 0; i < n - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    for (i = 0; i < n; i++)
        waitpid(pids[i], NULL, 0);

    printf("\nPipeline execution completed.\n");

    return 0;
}
```

Task 3 : To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```
vennela_07@VennelaPC:~$ cd ~
mkdir pipe_data_project
cd pipe_data_project
vennela_07@VennelaPC:~/pipe_data_project$ nano pipe_data.c
vennela_07@VennelaPC:~/pipe_data_project$ gcc -Wall -Wextra pipe_data.c -o pipe_data
vennela_07@VennelaPC:~/pipe_data_project$ ./pipe_data
Pipe created successfully.
Executing pipeline: ls | wc -l
Waiting for child processes...
2
Pipeline execution completed.
vennela_07@VennelaPC:~/pipe_data_project$ ls
pipe_data  pipe_data.c
```

GNU nano 7.2 pipe_data.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int fd[2];
    pid_t pid1, pid2;
    int status1, status2;

    /* Create a pipe */
    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipe created successfully.\n");

    /* Create the first child process */
    pid1 = fork();

    if (pid1 == -1)
    {
        perror("fork");
        close(fd[0]);
        close(fd[1]);
        return 1;
    }

    if (pid1 == 0)
    {
        /* First command: ls */
        close(fd[0]);

        if (dup2(fd[1], STDOUT_FILENO) == -1)
        {
            perror("dup2");
            _exit(1);
        }

        close(fd[1]);

        execlp("ls", "ls", (char *)NULL);
        perror("execlp ls");
        _exit(1);
    }

    /* Create the second child process */
    pid2 = fork();

    if (pid2 == -1)
    {
        perror("fork");
        close(fd[0]);
        close(fd[1]);
        waitpid(pid1, NULL, 0);
        return 1;
    }

    if (pid2 == 0)
    {
        /* Second command: wc -l */
        close(fd[1]);

        if (dup2(fd[0], STDIN_FILENO) == -1)
        {
            perror("dup2");
            _exit(1);
        }

        close(fd[0]);

        execlp("wc", "wc", "-l", (char *)NULL);
        perror("execlp wc");
        _exit(1);
    }

    /* Parent closes both pipe descriptors */
    close(fd[0]);
    close(fd[1]);

    printf("Executing pipeline: ls | wc -l\n");
    printf("Waiting for child processes...\n");
```

```
    /* Synchronize child processes */
    if (waitpid(pid1, &status1, 0) == -1)
        perror("waitpid first child");

    if (waitpid(pid2, &status2, 0) == -1)
        perror("waitpid second child");

    printf("Pipeline execution completed.\n");

    return 0;
}
```

Topic 4 : To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```
vennela_07@VennelaPC:~$ mkdir multiple_pipes.c
vennela_07@VennelaPC:~$ cd multiple_pipes.c
vennela_07@VennelaPC:~/multiple_pipes.c$ nano multiple_pipes.c
vennela_07@VennelaPC:~/multiple_pipes.c$ gcc multiple_pipes.c -o multiple_pipes
vennela_07@VennelaPC:~/multiple_pipes.c$ ./multiple_pipes
1
vennela_07@VennelaPC:~/multiple_pipes.c$ ls | grep .c | wc -l
1
```

```
 GNU nano 7.2                                                multiple_pipes.c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char *commands[][4] = {
        {"ls", NULL},
        {"grep", ".c", NULL},
        {"wc", "-l", NULL}
    };

    int n = 3;
    int pipes[2][2];
    pid_t pids[3];

    for (int i = 0; i < n - 1; i++) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            exit(1);
        }
    }

    for (int i = 0; i < n; i++) {
        pids[i] = fork();

        if (pids[i] == -1) {
            perror("fork");
            exit(1);
        }

        if (pids[i] == 0) {
            if (i > 0) {
                dup2(pipes[i - 1][0], STDIN_FILENO);
            }

            if (i < n - 1) {
                dup2(pipes[i][1], STDOUT_FILENO);
            }

            for (int j = 0; j < n - 1; j++) {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }
```

```
            }

            execvp(commands[i][0], commands[i]);
            perror("execvp");
            exit(1);
        }
    }

    for (int i = 0; i < n - 1; i++) {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    for (int i = 0; i < n; i++) {
        waitpid(pids[i], NULL, 0);
    }

    return 0;
}
```